
Practical - 2

Aim:

Implementation and Time analysis of linear and binary search algorithm.

Theory:

The Linear Search and Binary Search algorithms are the two fundamental techniques used to find a target value within a collection of data. While Linear Search checks elements sequentially and works on any dataset, Binary Search uses a divide-and-conquer strategy but strictly requires the data to be sorted beforehand.

Algorithm

Algorithm for Linear Search

Start.

Read the array A of n elements and the search element key.

Set $i = 0$.

Compare $A[i]$ with key. If they are equal, display the position and stop.

Otherwise, increment i and repeat Step 4 until $i = n$.

If the element is not found after checking all elements, display "Element not found".

Stop.

Algorithm for Binary Search

Note: Binary Search requires the array to be sorted.

Start.

Read the sorted array A of n elements and the search element key.

Initialize low = 0 and high = n - 1.

Find the middle element: $\text{mid} = (\text{low} + \text{high}) / 2$.

If $A[\text{mid}] == \text{key}$, display the position and stop. If $A[\text{mid}] < \text{key}$, set $\text{low} = \text{mid} + 1$; otherwise, set $\text{high} = \text{mid} - 1$.

Repeat Steps 4–5 until $\text{low} > \text{high}$. If the element is not found, display "Element not found"

Stop.

Program Code

```
#include <iostream>
```

```
#include <vector>
```

```
#include <chrono>
```

```
using namespace std;
```

```
using namespace std::chrono;
```

```
// Linear Search Function
```

```
int linearSearch(vector<int> arr, int key)
```

```
{
```

```
    for (int i = 0; i < arr.size(); i++)
```

```
    {
```

```
if (arr[i] == key)
    return i;
}
return -1;
}

// Binary Search Function
int binarySearch(vector<int> arr, int key)
{
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
```

else

high = mid - 1;

}

return -1;

}

int main()

{

int n = 100;

vector<int> arr(n);

// Creating a sorted array

for (int i = 0; i < n; i++)

arr[i] = i + 1;

int key;

cout << "Enter element to search: ";

cin >> key;

```
int index;
```

```
// Linear Search Timing
```

```
auto start = high_resolution_clock::now();
```

```
index = linearSearch(arr, key);
```

```
auto stop = high_resolution_clock::now();
```

```
cout << "\nLinear Search\n";
```

```
if (index != -1)
```

```
    cout << "Element found at index " << index << endl;
```

```
else
```

```
    cout << "Element not found\n";
```

```
cout << "Time Taken : "
```

```
    << duration_cast<microseconds>(stop - start).count()
```

```
    << " microseconds\n";
```

```
// Binary Search Timing
```

```
start = high_resolution_clock::now();
```

```
index = binarySearch(arr, key);

stop = high_resolution_clock::now();

cout << "\nBinary Search\n";

if (index != -1)
    cout << "Element found at index " << index << endl;
else
    cout << "Element not found\n";

cout << "Time Taken : "

    << duration_cast<microseconds>(stop - start).count()

    << " microseconds\n";

cout << "kolusu Uday Kiran\n";

cout << "92460118175\n";

cout << "5-En18\n";

return 0;

}
```

Result :

```
"C:\Users\kolus\Downloads\UDAY DAA-2.exe"
Enter element to search: 14

Linear Search
Element found at index 13
Time Taken : 253 microseconds

Binary Search
Element found at index 13
Time Taken : 71 microseconds
kolusu uday kiran
92460118175
5-En18

Process returned 0 (0x0)   execution time : 3.609 s
Press any key to continue.
```

